

UNIVERSIDAD

[Nombre de la universidad / facultad]

PROYECTO DE CURSO I-2026

Aula Virtual Académica

*Plataforma de aula virtual para el apoyo de procesos de enseñanza-aprendizaje en el sistema educativo
boliviano*

N.º de Grupo: [completar]

Integrante: Javi (Javi_jdev)

Docente: [completar nombre del docente]

Fecha de entrega: 28 de junio de 2026

1. Planteamiento del Problema y Justificación

1.1 Planteamiento del problema

Gran parte de las unidades educativas en Bolivia continúan gestionando de forma manual y dispersa actividades como la entrega de tareas, la publicación de material de apoyo, los anuncios del curso y, sobre todo, el registro de calificaciones por trimestre. Esta gestión manual genera pérdida de información, dificulta el seguimiento del avance de los estudiantes y consume tiempo administrativo que el docente podría destinar a la enseñanza.

Asimismo, las soluciones genéricas de aula virtual disponibles en el mercado (como Google Classroom) no están adaptadas a la estructura de evaluación por trimestres que utiliza el sistema educativo boliviano, lo que obliga a los docentes a llevar un registro paralelo de notas fuera de la plataforma.

1.2 Justificación

El presente proyecto busca resolver esta problemática mediante el desarrollo de “Aula Virtual Académica”, una aplicación web que centraliza la gestión de cursos, módulos de contenido, tareas, entregas, calificaciones y anuncios, incorporando el trimestre como unidad de organización académica propia del contexto boliviano. De esta manera se reduce el trabajo administrativo del docente, se ofrece a los estudiantes un canal único y ordenado de acceso a su información académica, y se sienta una base de software mantenible y simple, evitando una complejidad innecesaria en el modelo de evaluación.

2. Objetivo General y Objetivos Específicos

2.1 Objetivo general

Diseñar y desarrollar una aplicación web de aula virtual, utilizando Python y Flask bajo el patrón Application Factory, que permita a docentes y estudiantes gestionar cursos, contenidos, tareas, entregas y calificaciones de manera centralizada, organizada por trimestres.

2.2 Objetivos específicos

- Implementar un sistema de autenticación de usuarios con los roles de administrador, docente y estudiante.
- Desarrollar un panel administrativo con métricas básicas del uso de la plataforma (cantidad de usuarios, cursos, entregas, etc.).
- Diseñar un modelo de base de datos relacional normalizado que soporte cursos, módulos, recursos, tareas, entregas, calificaciones, anuncios e inscripciones.
- Implementar operaciones CRUD completas para cada uno de los módulos funcionales del sistema.
- Construir una interfaz de usuario responsiva utilizando Tailwind CSS, con un sistema de diseño en modo oscuro por defecto.
- Incorporar el trimestre académico como atributo central de las tareas y calificaciones, reflejando el sistema de evaluación boliviano.
- Desplegar la aplicación en un entorno de producción accesible públicamente.

3. Descripción de los Módulos Principales

El sistema está organizado siguiendo el patrón Application Factory de Flask, con blueprints independientes por módulo de dominio. A continuación se describe cada módulo funcional, su propósito y sus operaciones principales.

3.1 Autenticación y gestión de usuarios

Permite el registro e inicio de sesión de usuarios, con tres roles diferenciados: administrador, docente y estudiante. El administrador puede crear, editar, activar/desactivar y eliminar usuarios desde un panel dedicado. Las contraseñas se almacenan cifradas mediante Flask-Bcrypt y la sesión se gestiona con Flask-Login.

 [Captura de pantalla: formulario de inicio de sesión]

 [Captura de pantalla: listado/gestión de usuarios (vista administrador)]

3.2 Panel administrativo (Core)

Vista principal para el rol administrador con métricas básicas del sistema: número total de usuarios registrados por rol, número de cursos activos, número de entregas realizadas y otros indicadores de uso de la plataforma.

 [Captura de pantalla: panel administrativo con métricas]

3.3 Categorías y Cursos

Los cursos se organizan por categorías (por ejemplo: Matemáticas, Lenguaje, Ciencias). Cada curso es creado por un docente y genera automáticamente un código de acceso de 6 dígitos que los estudiantes utilizan para inscribirse, de forma similar a Google Classroom. El docente puede editar el curso, activarlo/desactivarlo y gestionar las inscripciones de sus estudiantes.

 [Captura de pantalla: listado de cursos]

 [Captura de pantalla: detalle de un curso con código de acceso]

3.4 Módulos y Recursos

Dentro de cada curso, el docente organiza el contenido en módulos (unidades temáticas), y dentro de cada módulo puede publicar recursos de apoyo (archivos descargables como PDFs, presentaciones o documentos).

 [Captura de pantalla: módulos de un curso con sus recursos]

3.5 Tareas y Entregas

El docente publica tareas dentro de un módulo, indicando título, instrucciones, fecha límite, puntaje máximo y el trimestre académico correspondiente (1º, 2º o 3º). Los estudiantes inscritos pueden subir su entrega como archivo adjunto antes de la fecha límite. El docente visualiza el listado de entregas por tarea, identificando además a los estudiantes que aún no han entregado.

 [Captura de pantalla: formulario de creación de tarea]

 [Captura de pantalla: listado de entregas de una tarea]

3.6 Calificaciones

Cada entrega puede ser calificada por el docente asignando una nota y una retroalimentación escrita. El sistema calcula automáticamente, para cada estudiante, el porcentaje de aprovechamiento por trimestre (suma de notas obtenidas sobre la suma de puntajes máximos posibles de las tareas de ese trimestre), permitiendo tanto al docente como al estudiante visualizar un resumen de notas por curso y por trimestre.

 [Captura de pantalla: formulario de calificación de una entrega]

 [Captura de pantalla: resumen de notas por trimestre (vista estudiante)]

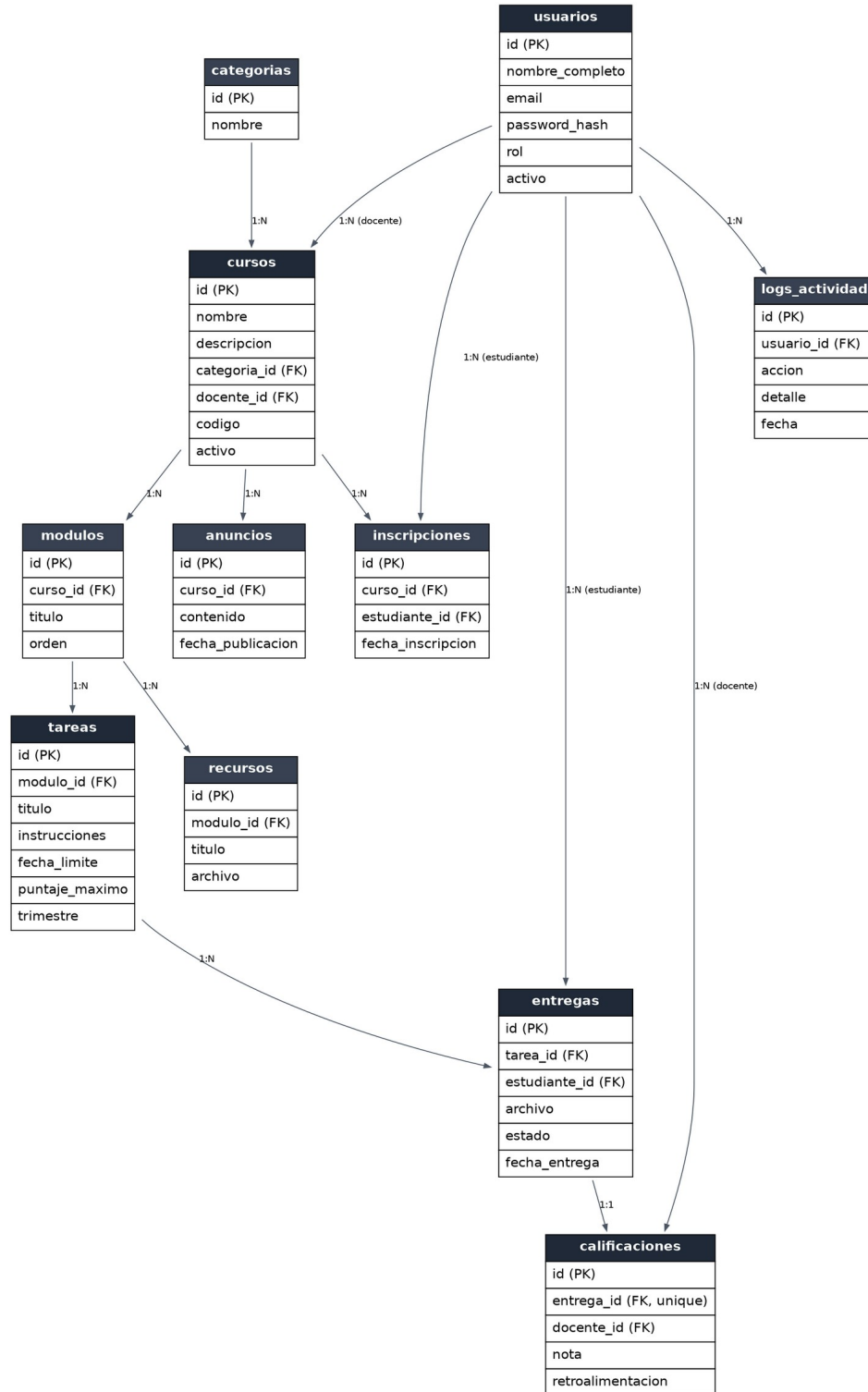
3.7 Anuncios

Los docentes pueden publicar anuncios dentro de un curso para comunicar información relevante a todos los estudiantes inscritos (cambios de fecha, recordatorios, indicaciones generales), visibles en la página de detalle del curso.

 [Captura de pantalla: anuncios publicados en un curso]

4. Modelo de Base de Datos

El modelo de datos está compuesto por 11 tablas relacionales, normalizadas y vinculadas mediante claves foráneas, gestionadas con SQLAlchemy y Flask-Migrate. A continuación se presenta el diagrama entidad-relación del sistema:



4.1 Descripción de las tablas principales

Tabla	Descripción
usuarios	Almacena administradores, docentes y estudiantes con su rol y credenciales.

logs_actividad	Bitácora de acciones relevantes realizadas por los usuarios (auditoría).
categorias	Clasificación temática de los cursos.
cursos	Curso dictado por un docente, con código de acceso para inscripción.
inscripciones	Relación N:M entre estudiantes y cursos.
modulos	Unidades temáticas dentro de un curso.
recursos	Archivos de apoyo publicados dentro de un módulo.
tareas	Actividad evaluada, con fecha límite, puntaje máximo y trimestre.
entregas	Archivo entregado por un estudiante para una tarea.
calificaciones	Nota y retroalimentación asociadas a una entrega.
anuncios	Comunicados publicados por el docente dentro de un curso.

5. Descripción de Tecnologías Utilizadas

Categoría	Tecnología	Uso en el proyecto
Backend	Python 3 / Flask	Framework principal, implementado con el patrón Application Factory (create_app).
Base de datos	SQLite + SQLAlchemy	ORM para el modelado de las 11 tablas relacionales, con back_populates en todas las relaciones.
Migraciones	Flask-Migrate (Alembic)	Control de versiones del esquema de base de datos.
Autenticación	Flask-Login + Flask-Bcrypt	Gestión de sesiones de usuario y cifrado de contraseñas.
Formularios	request.form (validación manual)	Procesamiento de formularios sin librerías adicionales, siguiendo el patrón sugerido por la cátedra.
Frontend	Jinja2 + Tailwind CSS	Plantillas del lado del servidor con un sistema de diseño en modo oscuro por defecto.
Control de versiones	Git / GitHub	Repositorio público del código fuente.
Despliegue	[Render / Heroku / otro]	Entorno de producción donde se publica la aplicación.

6. Uso de Herramientas de Inteligencia Artificial

6.1 Herramienta utilizada

Se utilizó Claude (Anthropic) como asistente de desarrollo durante distintas etapas del proyecto, principalmente para la revisión arquitectónica del código y para apoyar la documentación final.

6.2 Finalidad de uso

- Comparar la arquitectura del proyecto contra un proyecto de referencia ("InventaPro") entregado por la cátedra, identificando convenciones de blueprints, modelos ORM y manejo de formularios.
- Detectar funcionalidades sobredimensionadas respecto al nivel de complejidad esperado para el curso (sistema de dimensiones de evaluación, autoevaluación de estudiantes e historial de cambios de notas).
- Recibir una propuesta concreta de simplificación del módulo de calificaciones, manteniendo un rasgo distintivo del proyecto (la organización por trimestre, propia del sistema educativo boliviano).
- Aplicar la refactorización resultante directamente sobre el código fuente (modelos, rutas y plantillas), verificando que la aplicación siguiera funcionando sin errores de importación.
- Generar el diagrama entidad-relación y este informe final del proyecto.

6.3 Descripción general de las consultas realizadas

Las consultas se centraron en pedir una revisión comparativa del código contra el proyecto de referencia, solicitando observaciones sobre qué simplificar y por qué. A partir de esas observaciones, se solicitó un plan de cambios detallado (qué modelos, rutas y plantillas modificar y qué conservar), y posteriormente se solicitó la aplicación directa de dicho plan sobre el proyecto, así como su verificación final (búsqueda de referencias rotas y prueba de importación de la aplicación).

6.4 Reflexión sobre su aporte al desarrollo del proyecto

El uso de la herramienta de IA permitió identificar de forma objetiva qué partes del sistema agregaban complejidad innecesaria sin aportar valor académico adicional, comparando siempre contra un proyecto de referencia concreto en lugar de criterios subjetivos. Esto evitó eliminar funcionalidades core del proyecto y, en cambio, permitió enfocar la simplificación en el módulo que realmente la necesitaba (calificaciones), preservando el elemento diferenciador del proyecto: la organización académica por trimestres del sistema educativo boliviano. El resultado fue un código más mantenible, más cercano en tamaño y complejidad al proyecto de referencia, y verificado para asegurar que siguiera funcionando correctamente tras los cambios.

7. Conclusiones y Aprendizajes

- Se logró desarrollar una aplicación funcional de aula virtual que cubre el flujo completo de un curso: creación, inscripción, contenido, tareas, entregas, calificación y comunicación mediante anuncios.

- La comparación contra un proyecto de referencia resultó clave para calibrar el nivel de complejidad esperado, evitando tanto la sobre-ingeniería como la sub-implementación de funcionalidades.
- Incorporar el trimestre como atributo central, en lugar de un sistema completo de dimensiones de evaluación, permitió mantener el “toque boliviano” del proyecto sin comprometer su simplicidad.
- El trabajo con SQLAlchemy y back_populates en relaciones bidireccionales reforzó la comprensión del modelado relacional y de las buenas prácticas de ORM.
- Se reforzó la importancia de verificar la integridad de la aplicación (imports, rutas, plantillas) después de cualquier refactorización significativa, antes de considerar un cambio como terminado.

 [Agregar aquí cualquier aprendizaje personal adicional sobre Flask, despliegue o trabajo en equipo]